

Workshop 9: Processing data with Pandas

FIE463: Numerical Methods in Macroeconomics and Finance using Python

Richard Foltyn
NHH Norwegian School of Economics

March 19, 2026

See GitHub repository for notebooks and data:

<https://github.com/richardfoltyn/FIE463-V26>

Exercise 1: Data cleaning

Before doing actual data analysis, we usually first need to clean the data. This might involve steps such as dealing with missing values and encoding categorical variables as integers.

Load the Titanic dataset in `titanic.csv` and perform the following tasks:

1. Report the number of observations with missing Age, for example, using `isna()`.
2. Compute the average age in the dataset. Use the following approaches and compare your results:
 1. Use the `mean()` method.
 2. Convert the Age column to a NumPy array using `to_numpy()`. Experiment with NumPy's `np.mean()` and `np.nanmean()` to see if you obtain the same results.

3. Replace all missing ages with the mean age you computed above, rounded to the nearest integer. Convert this updated Age column to integer type using `astype()`.

Note that in “real” applications, replacing missing values with sample means is usually not a good idea.

4. Generate a new column `Female` which takes on the value one if `Sex` is equal to “female” and zero otherwise. This is called an *indicator* or *dummy* variable, and is preferable to storing such categorical data as strings. Delete the original column `Sex`.
5. Save your cleaned dataset as `titanic-clean.csv` using `to_csv()` with `,` as the field separator. Tell `to_csv()` to *not* write the DataFrame index to the CSV file as it’s not needed in this example.

Exercise 2: Daily returns of US stock market indices

In this exercise, we examine how the three major US stock market indices performed last year using data from Yahoo! Finance.

1. Use the `yfinance` library and its `download()` function to obtain the time series of daily observations for the [S&P 500](#), the [Dow Jones Industrial Average \(DJIA\)](#) and the [NASDAQ Composite](#) indices. Restrict the sample to the period from 2025-01-01 to 2025-12-31 and keep only the closing price stored in column `Close`.

Hint: The corresponding ticker symbols are `^GSPC`, `^DJI`, `^IXIC`, respectively.

2. Rename the DataFrame columns to 'SP500', 'Dow Jones' and 'NASDAQ' using the `rename()` method.

Hint: `rename(columns=dict)` expects a dictionary as an argument which maps existing to new column names.

3. Plot the three time series (one for each index) in a single graph. Label all axes and make sure your graph contains a legend.

Hint: You can directly use the `DataFrame.plot()` method implemented in pandas.

4. The graph you created in the previous sub-question is not well-suited to illustrate how each index developed in 2025 since the indices are reported on vastly different scales (the S&P 500 appears to be an almost flat line).

To get a better idea about how each index fared in 2025 relative to its value at the beginning of the year, normalize each index by its value on the first trading day in 2025 (which was 2025-01-02). Plot the resulting normalized indices.

5. For each index, compute the daily returns, i.e., the relative change vs. the previous closing price in percent. Create a plot of the daily returns for all indices.

Hint: Use `pct_change()` to compute the change relative to the previous observation.

Exercise 3: Daily returns of the Magnificent Seven

In this exercise, you are asked to analyze the weekly stock market returns of the so-called Magnificent Seven, which are some of the most successful tech companies of the last decades: Apple (AAPL), Amazon (AMZN), Google (GOOG), Meta (META), Microsoft (MSFT), Nvidia (NVDA), and Tesla (TSLA).

1. Load the CSV data from `../data/stockmarket/magnificent7.csv`. Inspect the first few rows to familiarize yourself with the columns present in the DataFrame.

Keep only the columns Date, Ticker, Open, and Close.

2. You want to compute weekly returns for each of the 7 stocks. To this end, you need to reshape the DataFrame so that Date is the index and the remaining dimensions are in (hierarchical) columns.

One way to achieve this is to use the `pivot()` function (also see the graphical illustration in the [user guide](#)). Call this function with the arguments `index='Date'` and `columns='Ticker'` and inspect the result.

This should generate a hierarchical column index with Open and Close at the top level.

Drop all rows with any missing values which arise because these stocks have been listed at different points in time.

3. Your data is now in a format that can be resampled to weekly frequency. Use `resample()` to convert the data to weekly observations.

Compute the weekly returns as the relative difference between the *first* Open quote and the *last* Close quote for each ticker in each week.

Hint: You should use `resample('W-MON')` so that the resampled weeks begin on Mondays (as opposed to the default Sundays).

Hint: For example, to select the first Open value in each week, you should use `resample('W-MON')['Open'].first()`.

4. Create a 3-by-3 figure and plot the weekly returns you computed for each ticker as a histogram, using 25 bins (i.e., `bins=25` should be passed to the `hist()` function).

Since you have only 7 tickers but 9 subplots in the figure, the last two remaining subplots should remain empty.

Hint: You can either use `DataFrame.hist()` to plot the histogram, or Matplotlib's `hist()` function. In either case, you should add `density=True` such that the histogram is appropriately rescaled and comparable to the normal density.

5. **[Advanced]** Compare the histograms you created to the normal (Gaussian) probability density function (PDF) to get an idea of how much weekly returns differ from a normal distribution.

First, compute mean and standard deviation for each ticker and tabulate these.

Then add a line showing the normal PDF to each of the return histograms you created previously, using the mean and standard deviation for each ticker.

Hint: Use the `pdf()` method of the `scipy.stats.norm` class to compute the normal density.

6. Finally, you are interested in how the weekly returns are correlated across the 7 stocks.

Create a figure with 7-by-7 subplots showing the pairwise correlations for each combination of stocks.

You can do this either with the `scatter_matrix()` function contained in `pandas.plotting`, or build the figure using Matplotlib functions.

[Advanced] Additionally, use the `DataFrame.corr()` method to compute the pairwise correlation matrix. Extract these values and add them as text to each of the 7-by-7 subplots (e.g., the correlation between returns on AAPL and AMZN is about 0.43, so this text should be added to the subplot showing the scatter plot of AAPL vs. AMZN).

Exercise 4: Business cycle correlations

Use the macroeconomic data from the folder `../data/FRED` to solve the following tasks:

1. There are seven decade-specific files named `FRED_monthly_19X0.csv` where X identifies the decade (X takes on the values 5, 6, 7, 8, 9, 0, 1). Write a loop that reads all seven files as DataFrames and stores them in a list.

Hint: Recall that you can use `pd.read_csv(..., index_col='DATE', parse_dates=['DATE'])` to automatically parse strings stored in the DATE column as dates.

2. Use `pd.concat()` to concatenate these datasets into a single DataFrame and make sure that DATE is set as the index.
3. You realize that your data does not include GDP since this variable is only reported at quarterly frequency. Load the GDP data from the file `GDP.csv` and merge it with your monthly data using an *inner join*.
4. You want to compute how (percent) changes of the variables in your data correlate with percent changes in GDP.
 1. Create a *new* DataFrame that contains the percent changes in CPI and GDP (using `pct_change()`), and the absolute changes for the remaining variables (using `diff()`).
 2. Compute the correlation of the percent changes in GDP with the (percent) changes of all other variables (using `corr()`). What does the sign and magnitude of the correlation coefficient tell you?

Exercise 5: Okun's Law

In this exercise, we investigate [Okun's Law](#) based on quarterly US data for each of the last seven decades.

Okun's Law relates unemployment to the output gap. One version (see Jones: Macroeconomics, 2019) is stated as follows:

$$\underbrace{u_t - \bar{u}_t}_{\text{cyclical unempl.}} = \alpha + \beta \underbrace{\left(\frac{Y_t - \bar{Y}_t}{\bar{Y}_t} \right)}_{\text{output gap}} \quad (3.1)$$

where u_t is the unemployment rate, $\bar{u}_t$ is the natural rate of unemployment, Y_t is output (GDP) and $\bar{Y}_t$ is potential output. We refer to $u_t - \bar{u}_t$ as “cyclical unemployment” and to the term in parentheses on the right-hand side as the “output gap.” Okun's Law says that the coefficient β is negative, i.e., cyclical unemployment is higher when the output gap is low (negative) because the economy is in a recession.

Use the FRED data in the `../..data/FRED` folder and perform the following tasks:

1. Load the time series stored in `GDP.csv` (real GDP), `GDPPOT.csv` (real potential GDP), `UNRATE.csv` (unemployment rate) and `NROU.csv` (noncyclical rate of unemployment), where the last series corresponds to the natural rate of unemployment mentioned above.

Combine these series into a single `DataFrame` so that each represents a column, and keep only observations from 1950-2019. The resulting data should be at quarterly frequency since GDP is only observed at these intervals.

Hint: Use `pd.read_csv(..., index_col='DATE', parse_dates=['DATE'])` to automatically parse strings stored in the `DATE` column as dates and set it as the index.

2. Compute the output gap and cyclical unemployment rate as defined above, and add them as columns to the `DataFrame`.

Plot these variables in a scatter plot with the output gap on the x -axis and the cyclical unemployment on the y -axis. Does Okun's Law hold over the sample period?

3. You wonder if the relationship has changed over the last decades. To answer this question, create a new column `Decade` which stores the decade of each observation, e.g., 1950, 1960, etc. Verify that each decade has 40 quarterly observations in your data.

Hint: Since you have a date index, the calendar year can be retrieved from the attribute `df.index.year`.

4. Create a figure with 3-by-3 subplots showing the same scatter plot as above, but separately for each decade. Since we have data for only 7 decades, the last two subplots should remain empty.
5. **[Advanced]** Write a function `regress_okun()` which accepts a `DataFrame` containing a decade-specific sub-sample as the only argument, and estimates the coefficients α (the intercept) and β (the slope) of the above regression equation (3.1).

This function should return a `Series` with two elements which store the intercept and slope.

To run the regression by decade, group the data by `Decade` and call the `apply()` method, passing `regress_okun` that you wrote as the argument.

Hint: Use NumPy's `lstsq()` to perform the regression. To regress the dependent variable y on regressors X , you need to call `lstsq(X, y)`. To include the intercept, you manually have to create X such that the first column contains only ones.

6. **[Advanced]** Plot your results: for each decade, create a scatter plot of the raw data and overlay it with the regression line you estimated.